Sanjivani Rural Education Society's

Sanjivani College of Engineering, Kopargaon-423603

(An Autonomous Institute Affiliated to Savitribai Phule Pune University, Pune) NAAC 'A' Grade Accredited

Department of Computer Engineering

Mini Project Report

Mini Project: RSA Cryptosystem

Title: RSA Encryption and Decryption Using Number Theory

(SY) B.Tech Computer Engineering

Under the Guidance of

Dr. S.N. Gunjal

DURING THE ACADEMIC YEAR 2026-27 (Semester-III)

Team Members

Roll No	Name	PRN No	Role in Team
53	Diksha Gunjal	0125UCSF1053	Team Member
54	Rutuja Bagul	0125UCSF1054	Team Member
55	Anushka Kale	0125UCSF1055	Team Member
56	Yuvraj Kurade	0125UCSF1056	Team Member

Abstract

RSA is one of the most widely used public-key (asymmetric) cryptosystems. Its security rests on the fact that multiplying two large primes is easy, while recovering them from their product is computationally hard. This project demonstrates the complete RSA workflow: choosing two primes, deriving the public and private keys using number theory, encrypting a text message character by character, and decrypting it back to the original. The system is implemented in C++ with input validation, Euclid's GCD algorithm, and modular exponentiation. A test run with p = 17 and q = 23 successfully encrypted the message "hello" and recovered it exactly. The report also discusses why this teaching version is not secure and what real-world RSA adds.

1. Introduction

RSA is named after its inventors Rivest, Shamir and Adleman. It is an **asymmetric** algorithm: it uses a pair of keys. The **public key** is shared openly and used to encrypt, while the **private key** is kept secret and used to decrypt. Because the two keys are mathematically related but not derivable from each other in practice, two parties can communicate securely without first sharing a secret.

1.1 Objectives

- Understand the working principle of RSA.
- Generate a public and private key pair from two primes.
- Encrypt a message using the public key.
- Decrypt the ciphertext back using the private key.
- Implement the whole process in C++.

2. Number Theory Background

RSA relies on a small set of number-theoretic ideas:

- **Prime numbers (p, q):** two distinct primes form the foundation of the keys.
- **Euler's totient:** for $n = p \times q$, $\varphi(n) = (p - 1)(q - 1)$.
- **Greatest common divisor:** the public exponent e must satisfy $GCD(e, \varphi(n)) = 1$.
- **Modular arithmetic:** all encryption and decryption is done modulo n.
- **Modular inverse:** the private exponent d satisfies $(d \times e) \bmod \varphi(n) = 1$.

3. Algorithm

3.1 Key Generation

1. Choose two distinct prime numbers p and q.
2. Compute $n = p \times q$.
3. Compute $\varphi(n) = (p - 1)(q - 1)$.

4. Choose e with $1 < e < \varphi(n)$ and $\text{GCD}(e, \varphi(n)) = 1$.
5. Compute d such that $(d \times e) \bmod \varphi(n) = 1$.
6. Public key = (e, n); private key = (d, n).

3.2 Encryption and Decryption

Each character is converted to its ASCII value M and encrypted as $\mathbf{C = M^e \bmod n}$. The receiver recovers it using $\mathbf{M = C^d \bmod n}$ and converts the number back to a character.

4. Worked Example

4.1 Key Generation with p = 17, q = 23

Quantity	Computation	Value
n	17 × 23	391
φ(n)	16 × 22	352
e	smallest value with GCD(e, 352) = 1	3
d	3 × d mod 352 = 1	235 (3 × 235 = 705 = 2 × 352 + 1)
Public key	(e, n)	(3, 391)
Private key	(d, n)	(235, 391)

4.2 Encrypting "hello"

For the letter 'h' (ASCII 104): $104^3 \bmod 391 = 348$. Decrypting, $348^{235} \bmod 391 = 104$, which is 'h'. Applying the same process to every character:

Char	ASCII (M)	Cipher $C = M^3 \bmod 391$	Decrypted
h	104	348	104 = h
e	101	16	101 = e
l	108	301	108 = l
l	108	301	108 = l
o	111	304	111 = o

Ciphertext: **348 16 301 301 304**. Decrypting returns **hello**. Note that both 'l' characters produce the same value, 301 (see Limitations).

5. Implementation in C++

The program is organised into small functions. Each is described below with its core code.

5.1 GCD Function

Implements the Euclidean algorithm and is used to confirm that e is relatively prime to φ(n). For example, GCD(3, 352) = 1.

```
int gcd(int a, int b)
{
   while (b != 0)
   {
      int temp = b;
      b = a % b;
      a = temp;
   }
   return a;
}
```

5.2 Modular Exponentiation

Computes $base^{exponent}$ mod n. The result is reduced modulo n at every multiplication, so intermediate values never become huge. It is used for both encryption and decryption.

```
long long powerMod(long long base, long long exponent, long long mod)
{
   long long result = 1;
   base = base % mod;
   while (exponent > 0)
   {
      result = (result * base) % mod;
      exponent--;
   }
   return result;
}
```

5.3 Input Validation

isPrime() rejects any non-prime input by trial division up to the square root. The program also checks that p ≠ q and that n > 255 so every ASCII character value is smaller than n and can be encrypted without ambiguity.

```
bool isPrime(int num)
{
   if (num < 2)
      return false;
   for (int i = 2; i * i <= num; i++)
      if (num % i == 0)
         return false;
   return true;
}
```

5.4 Key Generation

```
int n = p * q;
int phi = (p - 1) * (q - 1);

int e = 2;
while (e < phi && gcd(e, phi) != 1)   // first valid e
```

```
    e++;

int d = 1;
while ((d * e) % phi != 1)          // modular inverse of e
    d++;
```

The first loop finds the smallest e that is coprime to φ(n); the second searches for d with (d × e) mod φ(n) = 1.

5.5 Encryption and Decryption

```
// Encrypt
for (char ch : message)
{
    int m = (unsigned char)ch;
    cipher[len] = powerMod(m, e, n);
    cout << cipher[len] << " ";
    len++;
}

// Decrypt
for (int i = 0; i < len; i++)
    cout << (char)powerMod(cipher[i], d, n);
```

Each character is converted to its ASCII code, encrypted with the public key, and stored in an array. Decryption applies the private key to each stored value and casts the result back to a character.

6. Program Output

```
===== RSA ENCRYPTION AND DECRYPTION =====
Enter prime number p: 4
4 is not a prime number. Please enter a prime number.
Enter prime number p: 17
Enter prime number q: 23

RSA Parameters:
n = 391
phi(n) = 352
Public Key  = (3, 391)
Private Key = (235, 391)

Enter message: hello
Encrypted message: 348 16 301 301 304
Decrypted message: hello
```

The output confirms that invalid input (4) is rejected, keys are generated correctly, and the decrypted text exactly matches the original message.

7. Applications

- **Secure communication:** encrypting data sent over untrusted networks.

- **Digital signatures:** proving who sent a message and that it was not altered.
- **Authentication:** verifying identities of users and servers.
- **Key exchange:** safely sharing symmetric session keys.
- **Internet security:** HTTPS/TLS certificates, secure email and VPNs.

8. Limitations of This Implementation

- **Tiny primes:** n = 391 can be factored instantly by hand, so the keys offer no real security. Real RSA uses 2048-bit or larger keys.
- **No padding:** identical plaintext characters always give identical ciphertext (both 'l' characters become 301), which leaks patterns. Real RSA uses padding such as OAEP.
- **Slow modular exponentiation:** *powerMod* multiplies once per unit of the exponent. Square-and-multiply would need only about $\log_2$(exponent) steps.
- **Brute-force key search:** d is found by incrementing from 1, which is fine for small φ(n) but impractical for large values. The extended Euclidean algorithm is the standard method.
- **Small public exponent and integer types:** e = 3 with no padding is vulnerable, and 32-bit *int* values restrict the usable prime size.

9. Future Scope

- Use square-and-multiply for fast modular exponentiation.
- Use the extended Euclidean algorithm to compute d.
- Support big-integer arithmetic for large primes.
- Add padding (OAEP) and a larger standard public exponent such as 65537.
- Add digital-signature generation and verification.

10. Conclusion

This project showed how number theory underlies RSA. Primes and Euler's totient generate the keys, and modular arithmetic performs both encryption and decryption. The C++ program successfully validated input, generated a key pair, encrypted the message "hello" into **348 16 301 301 304**, and recovered the original text. Although the small key size and lack of padding make this version suitable only for learning, it clearly illustrates the principles that secure real-world RSA.

References

- R. L. Rivest, A. Shamir and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Communications of the ACM*, 21(2), 1978.
- W. Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson.

• K. H. Rosen, *Elementary Number Theory and Its Applications*, Pearson.